



Tecnicatura Universitaria
en Programación a Distancia

Programación II

Trabajo Final Integrador

Aplicación Java con relación 1→1 unidireccional + DAO + MySQL

Integrantes:

Chiaravalloti, Agustin – Comisión 2.

Diaz, Natalia – Comisión 12.

Etchevest, Jorgelina Carla – Comisión 2.

Enlace al video explicativo:

<https://drive.google.com/file/d/12oDpfyZZuNd3sJBUUctEzZQURck030-l/view?usp=sharing>

Introducción

En este trabajo desarrollamos una aplicación en Java basada en el dominio de Pedidos y Envíos, con el objetivo de integrar los contenidos principales de la materia Programación 2. La elección de este dominio nos permitió trabajar con una relación uno a uno, aplicar una arquitectura por capas y utilizar una base de datos MySQL para la persistencia de la información.

Objetivo

Desarrollar una aplicación en Java que permita gestionar pedidos y sus envíos utilizando una arquitectura por capas, una base de datos relacional y las herramientas y prácticas estudiadas en la materia.

Integrantes y roles

Integrante 1 – Chiaravalloti, Agustín

- Responsable del diseño de la base de datos en MySQL.
- Elaboración del diagrama UML del dominio y su relación 1→1.

Integrante 2 – Diaz, Natalia

- Desarrollo de los paquetes models, config, dao y service en Java.
- Implementación de las entidades, validaciones y lógica de negocio.

Integrante 3 – Etchevest, Jorgelina Carla

- Desarrollo del paquete main y construcción del menú interactivo.
- Pruebas generales de funcionamiento y carga de datos desde consola.

Todos los integrantes participaron de la elaboración del informe, conclusiones y documentación final del trabajo.

Elección del dominio y justificación

El dominio elegido fue Pedidos → Envíos, una relación 1 a 1 entre una entidad principal (Pedido) y una dependiente (Envío). Este dominio resulta adecuado porque es un caso habitual en sistemas de ventas y logística, lo que facilita aplicar de manera práctica los temas de la materia.

El modelo permite representar correctamente la cardinalidad pedida: un Pedido puede no tener Envío, pero un Envío siempre pertenece a un único Pedido. Además, ofrece reglas de negocio claras, como control de estados, validación de tracking único y fechas de entrega.

En la base de datos, la relación 1→1 se implementó con una clave foránea UNIQUE en la tabla envios, lo cual es más simple que usar claves primarias compartidas y permite crear primero el Pedido y luego asociarle su Envío.

El dominio también se adapta bien a la arquitectura del proyecto, ya que permite desarrollar entidades, validaciones, consultas SQL y pruebas de forma ordenada. Facilita operaciones como crear pedidos, asignar envíos, listar resultados y aplicar baja lógica, cumpliendo con todos los requisitos del TP.

Diseño: decisiones clave

El diseño del sistema se basó en decisiones que permitieron modelar correctamente la relación 1→1 y mantener un código claro y consistente.

• Clase base “Base”

Se creó una clase abstracta que aporta id y eliminado a todas las entidades, evitando duplicación y permitiendo implementar soft delete de manera uniforme.

• Relación 1→1 unidireccional

Pedido conoce a Envío, pero no al revés. Esto reduce acoplamiento y evita ciclos en métodos como toString() o equals(). Además, refleja el uso natural del sistema: trabajar desde el pedido hacia su envío.

• Clave foránea UNIQUE

La tabla envíos contiene pedido_id como UNIQUE. Esta solución es simple de implementar en MySQL, garantiza que cada Envío tenga un solo Pedido y permite crear el Envío después del Pedido.

• Enumeraciones (ENUM)

Se utilizaron enums para los estados y tipos (EstadoPedido, EstadoEnvío, TipoEnvío, EmpresaEnvío). Esto asegura valores válidos, evita errores de escritura y facilita las validaciones en Java.

• Constructores diferenciados

Cada entidad posee un constructor vacío para nuevas creaciones desde el menú y otro completo para reconstrucción desde la base de datos. Esto simplifica el trabajo de los DAOs y mantiene ordenado el ciclo de vida de los objetos.

• Soft delete aplicado a todas las entidades

El atributo eliminado permite ocultar registros en vez de borrarlos físicamente, manteniendo integridad referencial y permitiendo consultas históricas.

UML

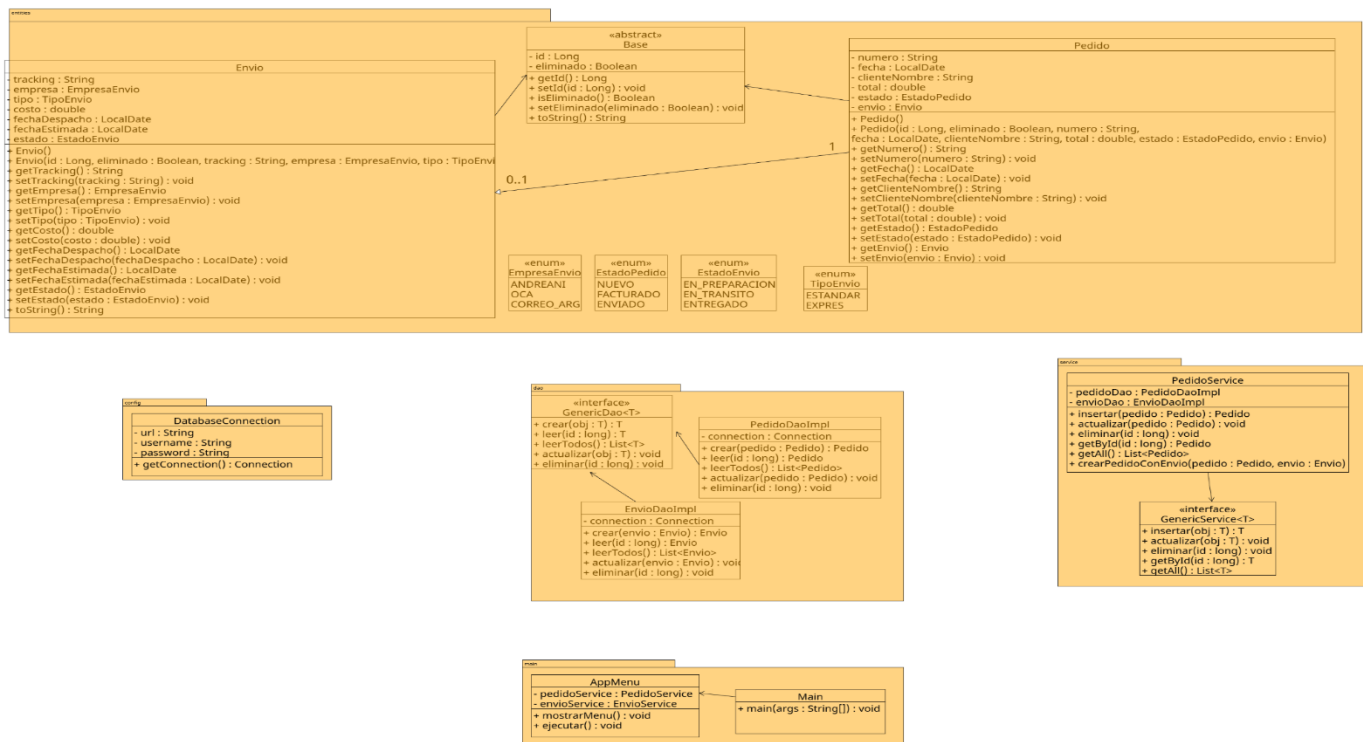
El diagrama UML modela el dominio Pedido → Envío, cumpliendo el requisito de una relación 1→1 unidireccional, donde la entidad Pedido contiene una referencia opcional a Envío, pero no al revés.

Organización por paquetes

El diseño sigue una arquitectura por capas, separada en paquetes:

- entities: modelos del dominio (Base, Pedido, Envío).
- entities.enums: tipos fijos controlados mediante enum.
- dao: interfaces y clases JDBC para persistencia.
- service: reglas de negocio y manejo transaccional
- config: clase de conexión a la base.
- main: menú de consola y entrada del programa.

Esta organización mejora la claridad, la cohesión y el mantenimiento del sistema.



Entidades del dominio

Las entidades Pedido y Envio heredan de la clase abstracta Base, que centraliza los atributos comunes id y eliminado (baja lógica). Pedido contiene información del cliente, monto, fecha y estado del pedido. Envio almacena datos logísticos: tracking, empresa, tipo, estado y fechas. Los valores controlados (estados, empresa, tipo) se representan mediante enumeraciones para evitar errores y mejorar la validez de los datos.

Capas DAO, Service y Main

Los DAO implementan la persistencia con JDBC usando PreparedStatement.

La capa Service gestiona validaciones y transacciones (commit/rollback).

El paquete main contiene AppMenu y Main, que interactúan únicamente con los services. La capa de presentación del sistema está organizada en cinco clases principales: Main, AppMenu, MenuDisplay, MenuHandler y TestConexion, cada una con una responsabilidad claramente definida para asegurar un flujo ordenado y coherente dentro de la aplicación.

- La clase Main actúa como punto de entrada del programa y se encarga de inicializar los componentes de interfaz y servicios, delegando la ejecución completa del menú sin contener lógica de negocio.
- La clase AppMenu administra el bucle principal de navegación, mostrando el menú general y dirigiendo las opciones seleccionadas hacia los controladores adecuados.
- MenuDisplay centraliza todas las operaciones de entrada y salida por consola, incluyendo lectura validada de datos, impresión de listados y mensajes informativos, evitando duplicación de código y proporcionando una interacción consistente.
- La clase MenuHandler implementa cada una de las acciones disponibles en el menú, coordinando la carga de datos ingresados por el usuario y llamando a los servicios correspondientes para crear, actualizar, buscar o eliminar Pedidos y Envíos; de este modo actúa como puente entre la interfaz y la capa de negocio.
- Finalmente, TestConexion permite verificar el acceso a la base de datos antes de ejecutar el sistema, asegurando que la configuración del entorno sea correcta.

- En conjunto, estas cinco clases se integran de forma directa con la capa Service, que contiene la lógica específica para cada operación, y con la capa DAO, responsable del acceso a la base MySQL a través de DatabaseConnection.

De esta manera, la estructura del MAIN garantiza una separación clara entre presentación, lógica de negocio y persistencia, permitiendo que todas las funcionalidades del TPI operen de forma coordinada y consistente.

Base de datos

Estructura de la base de datos

Para el dominio Pedido → Envío se diseñó la base de datos `tpi_pedidos_envios` en MySQL. Se definieron dos tablas principales: `pedido` (entidad A) y `envio` (entidad B). Cada tabla posee una clave primaria autoincremental (`id BIGINT`) y un campo eliminado `BOOLEAN` que implementa baja lógica.

La relación 1→1 unidireccional se implementó mediante una clave foránea única en la tabla `envio`: el campo `pedido_id` es `FOREIGN KEY` que referencia a `pedido(id)` y además es `UNIQUE`. De esta forma un pedido puede tener cero o un envío asociado, mientras que cada envío pertenece a un único pedido. También se definieron restricciones de unicidad sobre campos de negocio (`numero` en `pedido` y `tracking` en `envio`) y se utilizaron tipos `ENUM` para modelar los estados y opciones fijas (`estado` del `pedido`, `estado` del `envío`, `empresa` de `envío` y `tipo` de `envío`).

Se entregan dos scripts: uno para la creación de la base y las tablas (`01_creacion_bd.sql`) y otro para la carga de datos de prueba (`02_datos_prueba.sql`), que permiten levantar el proyecto desde cero y probar el CRUD desde el menú de consola.

Persistencia: estructura de la base, orden de operaciones y transacciones

La base de datos se diseñó en MySQL utilizando dos tablas principales: *pedidos* y *envios*, unidas por una relación 1→1 implementada mediante una clave foránea `UNIQUE` en la tabla *envios* (`pedido_id`). Ambas tablas incluyen el campo eliminado para soportar baja lógica sin comprometer la integridad referencial.

En cuanto al flujo de persistencia:

1. Se crea primero el Pedido (`INSERT` en `pedidos`).
2. Opcionalmente, se crea un Envío y luego se asocia mediante un `UPDATE` que asigna `pedido_id`.
3. Las consultas por listado y búsqueda filtran los registros con `eliminado = false`.

Las operaciones que combinan más de un DAO se ejecutan dentro de un Service que controla las transacciones. Allí se utiliza:

- `conn.setAutoCommit(false)` antes de iniciar una operación compleja.
- `commit()` cuando ambas acciones (Pedido y Envío, por ejemplo) se realizaron correctamente.
- `rollback()` si ocurre algún error, garantizando que la base no quede en un estado inconsistente.

Este esquema asegura integridad de datos y un manejo seguro de errores.

Validaciones y reglas de negocio

Las validaciones se implementan en la capa Service para separar la lógica de negocio del acceso a la base. Entre las principales reglas aplicadas:

- Tracking único para cada envío.
- Un pedido solo puede tener un envío asociado.
- El costo del envío debe ser mayor a cero.
- Las fechas deben tener coherencia (despacho y estimada válidas).
- Estados válidos únicamente a través de los enumerados (evita strings arbitrarios).
- En listados y búsquedas se excluyen los elementos con `eliminado = true`.

Estas validaciones garantizan coherencia, evitan cargas inválidas desde el menú y cumplen con los requisitos del dominio.

Pruebas realizadas

Para validar el correcto funcionamiento del sistema se ejecutó un conjunto de pruebas funcionales cubriendo todos los flujos principales del menú: creación de pedidos, registro de envíos, operación transaccional de alta conjunta, consultas por ID y por número, actualización de entidades, baja lógica y listados generales. Cada prueba se realizó ingresando datos válidos e inválidos para verificar tanto el comportamiento esperado como el manejo adecuado de errores. También se llevaron a cabo verificaciones directas en la base de datos mediante consultas SQL, confirmando la integridad de la información y el cumplimiento de la relación 1→1 entre Pedido y Envío. Los resultados obtenidos demostraron que el sistema gestiona correctamente las operaciones CRUD, respeta las restricciones de negocio y mantiene la consistencia de los datos incluso en escenarios de fallo, como en operaciones transaccionales. En conjunto, las pruebas realizadas evidencian que la aplicación cumple satisfactoriamente con los requisitos planteados en la consigna.

Pruebas ejecutadas (según el plan de pruebas):

1. Crear Pedido sin Envío.
2. Crear Envío asociado a un Pedido existente.
3. Crear Pedido + Envío en una sola transacción.
4. Listar todos los Pedidos activos.
5. Buscar Pedido por ID.
6. Buscar Pedido por Número.
7. Actualizar un Pedido.
8. Actualizar un Envío.
9. Eliminar Pedido (baja lógica).
10. Listar todos los Envíos activos.

```
Probando conexion a la base...
Conexion OK.
```

```
===== MENU PRINCIPAL =====
```

```
1 - Crear Pedido
2 - Crear Envio para Pedido
3 - Crear Pedido + Envio
4 - Listar Pedidos
5 - Buscar Pedido por ID
6 - Buscar Pedido por Numero
7 - Actualizar Pedido
8 - Actualizar Envio
9 - Eliminar Pedido
10 - Listar Envios
0 - Salir
```

```
Opcion: 1
```

```
Numero: P-1005
```

```
Fecha (yyyy-MM-dd): 2025-11-16
```

```
Cliente: Jorgelina Etchevest
```

```
Total: 28000
```

```
Pedido creado.
```

```
===== MENU PRINCIPAL =====
```

```
1 - Crear Pedido
2 - Crear Envio para Pedido
3 - Crear Pedido + Envio
4 - Listar Pedidos
5 - Buscar Pedido por ID
6 - Buscar Pedido por Numero
7 - Actualizar Pedido
8 - Actualizar Envio
9 - Eliminar Pedido
10 - Listar Envios
0 - Salir
```

```
Opcion: 2
```

```
ID del pedido: 6
```

```
Tracking: TRK-006
```

```
Seleccione empresa de envio:
```

```
1) ANDREANI
2) OCA
3) CORREO_ARG
```

```
Elija opcion: 2
```

```
Seleccione tipo de envio:
```

```
1) ESTANDAR
2) EXPRES
```

```
Elija opcion: 2
```

```
Costo: 12500
```

```
Fecha despacho: 2025-11-17
```

```
Fecha estimada: 2025-11-28
```

```
Envio creado y asociado.
```

```
===== MENU PRINCIPAL =====
```

```
1 - Crear Pedido
2 - Crear Envio para Pedido
3 - Crear Pedido + Envio
4 - Listar Pedidos
5 - Buscar Pedido por ID
6 - Buscar Pedido por Numero
7 - Actualizar Pedido
8 - Actualizar Envio
9 - Eliminar Pedido
10 - Listar Envios
0 - Salir
```

```
Opcion: 3
```

```
Numero: P-1003
```

```
Fecha (yyyy-MM-dd): 2025-11-16
```

```
Cliente: Claudio Ramirez
```

```
Total: 18500
```

```
¿Crear tambien el envio? (S/N): s
```

```
Tracking: TRK-2003
```

```
Seleccione empresa de envio:
```

```
1) ANDREANI
2) OCA
3) CORREO_ARG
```

```
Elija opcion: 1
```

```
Seleccione tipo de envio:
```

```
1) ESTANDAR
2) EXPRES
```

```
Elija opcion: 2
```

```
Costo: 15200
```

```
Fecha despacho: 2025-11-18
```

```
Pedido creado correctamente con envio asociado.
```

===== MENU PRINCIPAL =====

- 1 - Crear Pedido
- 2 - Crear Envio para Pedido
- 3 - Crear Pedido + Envio
- 4 - Listar Pedidos
- 5 - Buscar Pedido por ID
- 6 - Buscar Pedido por Numero
- 7 - Actualizar Pedido
- 8 - Actualizar Envio
- 9 - Eliminar Pedido
- 10 - Listar Envios
- 0 - Salir

Opcion: 4

```
Pedido{id=1, numero='P-0001', total=15000.0, estado='NUEVO', envio=TRK-001, eliminado=false}
Pedido{id=2, numero='P-0002', total=23000.5, estado='FACTURADO', envio=TRK-OCA-0002, eliminado=false}
Pedido{id=3, numero='P-0003', total=8000.0, estado='ENVIADO', envio=N/A, eliminado=false}
Pedido{id=5, numero='P-1002', total=21000.0, estado='ENVIADO', envio=N/A, eliminado=false}
Pedido{id=6, numero='p-2001', total=20000.0, estado='NUEVO', envio=TRK-006, eliminado=false}
Pedido{id=7, numero='P-1005', total=28000.0, estado='NUEVO', envio=N/A, eliminado=false}
Pedido{id=8, numero='P-2003', total=18500.0, estado='NUEVO', envio=TRK-2003, eliminado=false}
```

===== MENU PRINCIPAL =====

- 1 - Crear Pedido
- 2 - Crear Envio para Pedido
- 3 - Crear Pedido + Envio
- 4 - Listar Pedidos
- 5 - Buscar Pedido por ID
- 6 - Buscar Pedido por Numero
- 7 - Actualizar Pedido
- 8 - Actualizar Envio
- 9 - Eliminar Pedido
- 10 - Listar Envios
- 0 - Salir

Opcion: 5

ID: 1

```
Pedido{id=1, numero='P-0001', total=15000.0, estado='NUEVO', envio=TRK-001, eliminado=false}
```

===== MENU PRINCIPAL =====

- 1 - Crear Pedido
- 2 - Crear Envio para Pedido
- 3 - Crear Pedido + Envio
- 4 - Listar Pedidos
- 5 - Buscar Pedido por ID
- 6 - Buscar Pedido por Numero
- 7 - Actualizar Pedido
- 8 - Actualizar Envio
- 9 - Eliminar Pedido
- 10 - Listar Envios
- 0 - Salir

Opcion: 6

Numero: P-0001

```
Pedido{id=1, numero='P-0001', total=15000.0, estado='NUEVO', envio=TRK-001, eliminado=false}
```


===== MENU PRINCIPAL =====

1 - Crear Pedido
2 - Crear Envio para Pedido
3 - Crear Pedido + Envio
4 - Listar Pedidos
5 - Buscar Pedido por ID
6 - Buscar Pedido por Numero
7 - Actualizar Pedido
8 - Actualizar Envio
9 - Eliminar Pedido
10 - Listar Envios
0 - Salir

Opcion: 9

ID del pedido: 1

¿Esta seguro? (S/N): s

Pedido eliminado.

===== MENU PRINCIPAL =====

1 - Crear Pedido
2 - Crear Envio para Pedido
3 - Crear Pedido + Envio
4 - Listar Pedidos
5 - Buscar Pedido por ID
6 - Buscar Pedido por Numero
7 - Actualizar Pedido
8 - Actualizar Envio
9 - Eliminar Pedido
10 - Listar Envios
0 - Salir

Opcion: 10

```
Envio{id=1, tracking='TRK-AND-0001', empresa='ANDREANI', estado='EN_TRANSITO', eliminado=false}
Envio{id=2, tracking='TRK-OCA-0002', empresa='OCA', estado='ENTREGADO', eliminado=false}
Envio{id=3, tracking='TRK-001', empresa='ANDREANI', estado='EN_PREPARACION', eliminado=false}
Envio{id=4, tracking='TRK-200', empresa='CORREO_ARG', estado='EN_PREPARACION', eliminado=false}
Envio{id=5, tracking='TRK-006', empresa='OCA', estado='EN_PREPARACION', eliminado=false}
Envio{id=7, tracking='TRK-2003', empresa='ANDREANI', estado='EN_PREPARACION', eliminado=false}
```

Conclusiones y mejoras futuras

El desarrollo permitió aplicar todos los conceptos del programa: diseño de entidades, relación 1→1, JDBC, arquitectura en capas, validaciones, soft delete y manejo de transacciones. El dominio elegido (Pedido → Envío) resultó claro, práctico y fácil de extender.

Como mejoras futuras podrían incorporarse:

- interfaz gráfica o API REST,
- autenticación de usuarios,
- historial de estados del envío,
- reportes y consultas avanzadas,
- uso de connection pooling en la capa config.

Estas extensiones permitirían escalar el proyecto hacia un sistema más completo.

Como mejora de arquitectura, se podría integrar un connection pool mediante la librería HikariCP, ampliamente utilizada en entornos productivos por su eficiencia y bajo tiempo de respuesta. HikariCP permite administrar un conjunto de conexiones reutilizables hacia la base de datos, evitando la creación y cierre constante de conexiones individuales, lo cual reduce de manera significativa el costo computacional asociado al acceso JDBC tradicional. Aunque el trabajo práctico funciona correctamente con DriverManager, incorporar un pool ofrece una gestión más robusta de recursos, optimiza el manejo concurrente y sigue buenas prácticas recomendadas para aplicaciones de mayor escala. Esta integración no modifica la lógica del sistema ni las interfaces existentes, ya que getConnection() continúa retornando un objeto Connection, pero introduce un nivel adicional de rendimiento y estabilidad propio de aplicaciones profesionales.

Fuentes y herramientas utilizadas

- Material teórico y ejercicios del Campus Virtual de la Tecnicatura Universitaria en Programación – UTN, correspondientes a la materia Programación 1 (Año 2025).
- Videos explicativos de la misma cátedra.
- Documentación oficial de Java y MySQL.
- ChatGPT como apoyo para revisión de código.